

ARC-AGI-2 Agentic Solver Test Suite

Testing and Validation Project

Nagy Dezső
Számítástechnika IV.
Sapientia

April 2026

Contents

1	Introduction	2
1.1	What is software testing?	2
2	Project Overview	2
2.1	The ARC-AGI-2 benchmark	2
2.2	The base repo and what I changed	2
2.3	Why testing is needed	3
2.4	Architecture	3
3	Testing Framework	3
3.1	Pytest	3
3.2	Installation	3
4	Repository structure	4
4.1	Puzzle Data	4
4.2	Test directory layout	4
4.3	Shared fixtures	4
5	Testing Suite	5
5.1	Unit tests	5
5.2	Functional tests	6
5.3	Integration tests	7
6	Continuous integration	8
7	Summary	8

1 Introduction

1.1 What is software testing?

Testing is the practice of running a program against inputs you control and comparing what comes out against what you expected. It is not proof of correctness, a passing suite only says tests our expected assumptions on how the system should behave, but it is the cheapest way to catch the regressions that creep in between changes. When the project starts touching real APIs that cost money, that cheap feedback loop quickly becomes the difference between a solver you can add features to or one nobody dares to touch.

2 Project Overview

2.1 The ARC-AGI-2 benchmark

ARC-AGI-2 [1] is a visual reasoning benchmark. Each task gives the solver a handful of training pairs of small grids of integers (values 0–9) and asks it to figure out the transformation rule that maps input grids to output grids. The solver then has to apply that rule to a held-out test grid. The grids are small but the rules are diverse: rotations, recolouring, object isolation, symmetry completion, and many things in between. Performance is measured by how many test grids are solved correctly.

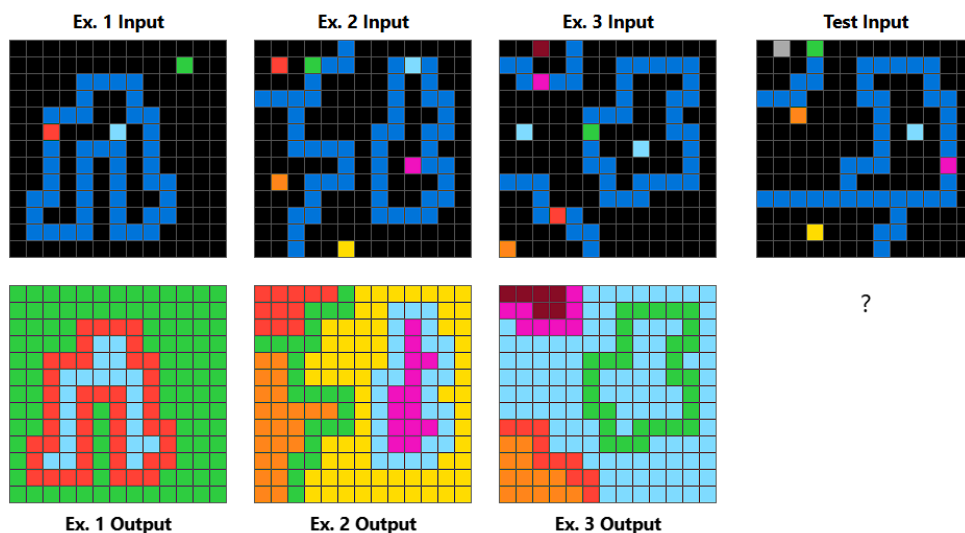


Figure 1: An example of an ARC-AGI task.

2.2 The base repo and what I changed

The codebase started as a fork of Confluence Labs’ *Saturating ARC-AGI-2* [2], which holds one of the best performing solvers on the public-evaluation set. That gave me a working solver to build on top of. The pieces I contributed and which the test suite focuses on are:

- A pluggable CLI layer: a common BaseCLI protocol with three concrete adapters (Gemini, OpenCode, Junie), so swapping the agent CLI is a single argument change.
- A Docker sandbox sitting alongside the original E2B runner under a shared SandboxRunner protocol.
- A typed model layer (Pydantic v2) and a refactored, typed orchestrator API.
- A streaming log protocol that splits agent stdout into a session log, a transcript, and a human-readable Markdown rendering, with per-call cost tracking.
- The full test suite with multiple tiers of unit, functional and integration tests.

2.3 Why testing is needed

The solver assigns multiple agents for each task, each running in its own sandbox, each calling a remote LLM API dozens of times. A single full evaluation run hits the API budget hard. Without tests, every refactor is a coin flip: you might find out at hour eight that a typo in the cost aggregator silently double-counted half the runs. With tests, the parser, the diff formatter, the orchestrator’s resume logic, and the sandbox’s output routing are all pinned down before any of that runs. The functional layer in particular catches the kind of bug that only shows up under concurrency: ordering issues, missing semaphore acquisitions, exception handling that drops one agent and crashes the rest.

2.4 Architecture

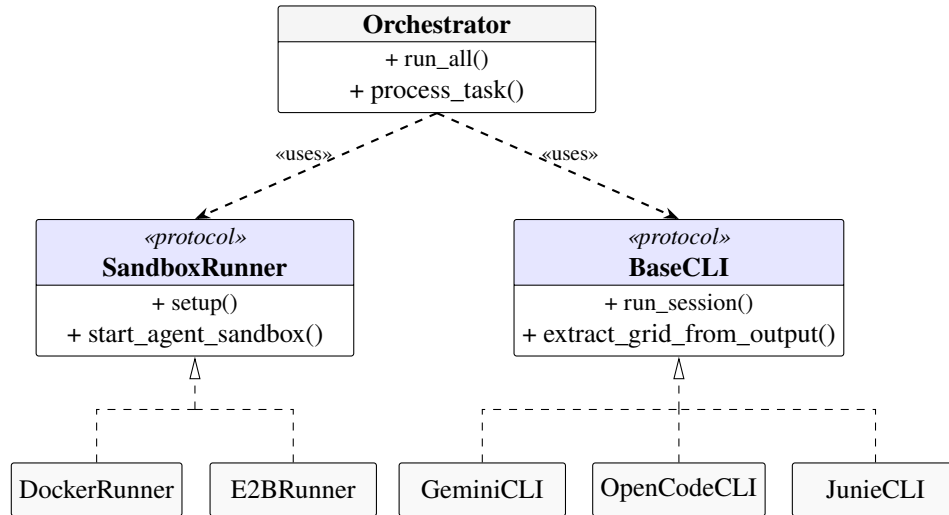


Figure 2: Class diagram of the pluggable layers.

At runtime, after parsing the command line arguments, the Orchestrator schedules one coroutine per ARC task. Each of them assigns agents through a API call semaphore, and every agent goes through the chosen **SandboxRunner** into a sandbox where the agent runner controls the chosen CLI for some number of refinement turns.

3 Testing Framework

3.1 Pytest

I picked `pytest` for the following reasons:

- It’s easy to use. Plain `assert` statements, and the test body reads like the hypothesis it encodes.
- It’s one of the most widely used Python testing libraries, and a particularly natural fit for a project this size. It has support for asynchronous tests with `pytest-asyncio` and mocking with `pytest-mock` so I didn’t have to write any boilerplate code.
- Fixtures take the pain out of test environment setup. Defining a temporary workspace, a mock task JSON, and a fully wired-up mock CLI in one file once and using them in later tests rather than copy-pasting setup code into every file.
- I like how the decorators make intent visible at a glance. `@pytest.mark.asyncio`, `@pytest.mark.parametrize`, and the `unit / functional / integration` markers all show up next to the test they describe, not in some external configuration.

3.2 Installation

Clone, install dev dependencies, run the suite:

```
git clone https://github.com/NagyDezso/arc-agi-2.git
cd arc-agi-2
uv sync --group dev

uv run pytest tests/unit
uv run pytest tests/functional
uv run pytest tests/integration # needs API keys and Docker access
```

4 Repository structure

The repository structure is as follows:

```
arc-agi-2/
|-- src/                # implementation
|-- tests/              # the suite
|-- data/               # ARC-AGI-2 puzzle JSONs
|-- pyproject.toml      # pytest + ruff config
|-- .github/workflows/tests.yml # CI
|-- main.py             # entry point
```

Listing 1: Top-level layout.

4.1 Puzzle Data

The `data/` folder holds two JSON files from the ARC Prize: the challenges file and the matching solutions file (`arc-agi_evaluation_challenges.json` and `arc-agi_evaluation_solutions.json`). The schema, in one line:

```
{ "<task_id>": { "train": [ { "input": [...], "output": [...] } ],
                  "test": [ { "input": [...] } ] }
```

Listing 2: Task schema.

Every input and output is a 2D list of integers in the range 0–9. The solver only sees the train pairs and the test inputs; the solutions file is only used at scoring time.

For testing, none of this real data is required. Tests build minimal task dictionaries on the fly via the `mock_task_data` fixture, with a single 2×2 train pair and a single 2×2 test input. That is enough to exercise the agent loop, the orchestrator state machine, and every code path that depends on task structure.

4.2 Test directory layout

```
tests/
|-- confest.py          # shared fixtures
|-- unit/
|   |-- test_agent_runner.py
|   |-- test_cli_impl.py
|   |-- test_docker_runner.py
|-- functional/
|   |-- test_agent_runner_errors.py
|   |-- test_orchestrator.py
|-- integration/
|   |-- test_cli_integration.py
|   |-- test_docker_integration.py
```

Listing 3: Tests groupings

4.3 Shared fixtures

`tests/confest.py` defines five fixtures every test in the suite can pull in:

- `mock_task_data`: a minimal task dict.
`"train": [{"input": [[0, 0], [0, 0]], "output": [[1, 1], [1, 1]]}`
`"test": [{"input": [[0, 0], [0, 0]]}]`.
- `temp_workspace`: a `tempfile.TemporaryDirectory` yielded as a `Path`, cleaned up automatically.
- `mock_raw_task_file`: writes `mock_task_data` to a `task.json` inside `temp_workspace` and returns the path.
- `mock_test_prompt`: a constant string used by tests that need a non-empty prompt.
- `mock_cli_impl`: a `MagicMock` configured with defaults (zero cost, empty session output).

5 Testing Suite

5.1 Unit tests

Pure functions and small components, no I/O beyond a temporary file.

5.1.1. Diff formatter: The function the agent loop uses to know what went wrong on its last attempt.
`test_format_diff`

Description: Verifies the diff formatter highlights mismatched cells correctly.

Input: Two grids differing in one cell.

Expected: Diff string naming the coordinate `((0, 1): 2 -> 0)` and printing both grids.

`test_format_diff_shape_mismatch`

Description: Verifies the diff formatter handles shape mismatches gracefully instead of crashing on a per-cell diff.

Input: Grids of different shapes.

Expected: A clear shape-mismatch message, no per-cell diff attempted.

5.1.2. Transform validator: Decides whether the agent's submitted Python passes the training examples.

`test_run_transform_success`

Description: Verifies that a correct transform passes against scripted training pairs.

Input: Correct `transform.py` function and matching training examples.

Expected: `(True, "All training examples pass.", fn)`.

`test_test_transform_failure`

Description: Verifies that a wrong transform produces the feedback string the agent loop echoes back to the LLM.

Input: A wrong `transform.py` function.

Expected: `(False, msg, None)` where `msg` starts with `Value mismatch`.

5.1.3. CLI output parsers: Pulls the final grid out of three different agent-CLI transcript formats.

`test_gemini_extract_grid_from_output`

Description: Verifies the Gemini parser pulls a grid out of both event types it can appear in.

Input: Gemini `tool_result` or `write_file` carrying a grid.

Expected: Parsed Python list.

`test_opencode_extract_grid_from_output`

Description: Verifies the OpenCode parser handles its different event format.

Input: OpenCode text event or `write` tool call carrying a grid.

Expected: Parsed Python list.

5.1.4. Readable-log writers: Builds a Markdown output of each agent run.

`test_gemini_write_readable_log_handles_split_deltas`

Description: Verifies streamed delta messages get stitched back into one piece of text.

Input: Two delta-message events "Hello " and "world".

Expected: Buffer holds exactly "Hello world", no duplicates.

`test_opencode_write_readable_log_includes_tool_result`

Description: Verifies tool calls render with the full Markdown layout (header and result block).

Input: OpenCode tool_use for bash.

Expected: Markdown contains `**Tool: bash**` and `**Tool Result:**`.

5.1.5. Docker output routing: Splits agent stdout into the right files.

`test_docker_runner_routes_status_and_transcript_lines`

Description: Verifies each kind of stdout line lands in its intended destination.

Input: Status and Transcript events plus an unknown-type line.

Expected: Status hits session log and logger, Transcript hits transcript file with no envelope, unknown lines are logged as warnings.

5.2 Functional tests

Real orchestrator and agent runner, mocked external dependencies (Docker, HTTP, the LLM CLI). These tests cover the multi-agent state machine.

5.2.1. Agent-loop error handling:

`test_run_agent_should_stop_on_fatal_errors`

Description: Verifies the agent loop aborts immediately on fatal errors instead of burning its full iteration budget.

Input: 5 parametrised fatal errors (`ModelNotFoundError`, Invalid model, `QuotaExceeded`, Access denied, Requested entity was not found).

Expected: `run_session` called exactly once, error preserved in `stderr`.

5.2.2. Score aggregation:

`test_process_task_aggregates_multi_agent_multi_test_results`

Description: Verifies cost, token and elapsed-time aggregation across multiple agents and tests.

Input: 2 agents × 2 tests, one empty grid.

Expected: Correct sum of costs / tokens / elapsed time, `score.submitted == 2`, persisted JSON has all four agent IDs.

`test_process_task_whole_task_counts_submissions_from_multiple_test_indexes`

Description: Verifies whole-task mode counts attempts spanning distinct test indices.

Input: Whole-task mode with attempts spanning two `test_index` values.

Expected: `score.submitted == 2`, `readable.md` generated.

`test_process_task_handles_empty_attempts`

Description: Verifies an empty attempt list is not counted as a submission.

Input: Agent returns `attempts=[]`.

Expected: `score.submitted == 0`, `sandbox` still called once.

5.2.3. Failure isolation:

`test_process_task_persists_exception_as_agent_error`

Description: Verifies a single failing agent does not break its sibling agents.

Input: One agent raises `RuntimeError`.

Expected: Failed agent recorded with error field, `error.log` written, surviving agent unaffected.

`test_run_all_continues_after_process_task_crash`

Description: Verifies a single task crash does not abort the entire run.

Input: `process_task` raises for one task.

Expected: Run finishes, `summary.json` contains only the successful task.

5.2.4. Concurrency & resume:

`test_process_task_respects_sandbox_semaphore_concurrency_limit`

Description: Verifies the concurrency semaphore actually serialises agent dispatch.

Input: 3 agents through a semaphore with a limit of 1.

Expected: `sandbox.max_active == 1` (serialised).

`test_run_all_skips_completed_tasks_and_writes_summary`

Description: Verifies resume logic skips already-finished tasks and still reports them in the summary.

Input: Resume dir already contains `task_a.json`, list `[a,b,c]`, `limit=1`.

Expected: Only `task_b` processed, `summary.json` reports all three in totals.

5.2.5. End-to-end (mock sandbox):

`test_process_task_integration`

Description: Smoke-tests the full pipeline against a mock sandbox, checking every artefact lands on disk.

Input: `MockSandbox` returns one attempt.

Expected: `score.submitted == 1`, run dir contains `task_results/*.json` and per-agent logs (`session.log`, `transcript.jsonl`, `readable.md`, `attempts.jsonl`).

5.3 Integration tests

Real credentials, real services. Skipped in CI. Can be run locally with provided API keys to detect external changes.

5.3.1. API smoke tests:

`test_gemini_real_usage`, `test_opencode_real_usage`, `test_junie_real_usage`

Description: Verifies each adapter actually talks to its live API.

Input: Real workspace and a live API call.

Expected: `turns > 0` and a non-zero token count.

5.3.2. Bad-model failure:

`test_gemini_bad_model_failure`, `test_opencode_bad_model_failure`

Description: Verifies adapters fail gracefully on an invalid model rather than throwing.

Input: Invalid model name.

Expected: Graceful return, error in `stderr`, no exception escapes the adapter.

5.3.3. Docker integration:

`test_docker_opencode_agent`

Description: Verifies a real Docker container can run an OpenCode session end-to-end.

Input: Real container running an OpenCode session.

Expected: `session.log` exists and contains the start marker.

`test_docker_run_agent_failure`

Description: Verifies the Docker sandbox handles a misconfigured run without crashing.

Input: Real container with a bad config.

Expected: No crash, no attempts submitted, error reaches `stderr`.

6 Continuous integration

The project has a CI workflow that runs on every push and every pull request against main. If the testing suite fails, the build will fail as well.

```
name: tests
on:
  push:          { branches: [main] }
  pull_request: { branches: [main] }

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v3
      - run: uv sync --group dev
      - run: uv run ruff check .
      - run: uv run pytest tests/unit tests/functional
      # Integration tests are intentionally skipped as they need API keys and
      # Docker access
```

Listing 4: `.github/workflows/tests.yml` (abbreviated).

7 Summary

The test suite covers the parts of the system where regressions are silent and expensive: the diff formatter that decides whether an agent’s transform is correct, the parsers that pull a grid out of three different CLI output formats, the orchestrator’s state machine across multi-agent and multi-test inputs, the resume logic that picks up partial runs, and the Docker sandbox’s output routing. The functional tests in particular cover concurrency-sensitive code paths: semaphore contention, exception propagation, partial failures. Preventing these errors reduces the risk of failed runs, that could be costly.

References

- [1] ARC Prize 2025. <https://arcprize.org/>
- [2] Confluence Labs, *Saturating ARC-AGI-2*. <https://github.com/confluence-labs/arc-agi-2>